some publi before we start :P

@MATH_CLUB_IITM

checkout the website too!

https://mathiitm.com

some publi before we start :P

PROGRAMMINGCLUB_IITM

checkout the website too! https://progclubiitm.com/

HI KIDS,

LET'S START OFF WITH A SMALL PUZZLE,

SHALL WE?

HI KIDS,
LET'S START OFF WITH A SMALL PUZZLE,
SHALL WE?

IT'S THE YEAR 2050 AND TWO OF THE BEST MINDS IN THE WORLD - THE SIDDHARTHS STUMBLED UPON THE GREATEST INVENTION IN THE HISTORY OF MANKIND - A MULTIVERSAL WINDOW.

A MULTIVERSAL WINDOW COUNTS ALL THE POSSIBLE BRANCHES OF AN EVENT STARTING FROM WHEN THE DEVICE IS ACTIVATED

USUALLY THE AMOUNT OF BRANCHING AT ANY GIVEN INSTANT IN THIS WORLD WOULD BE SO HUGE THAT THE DEVICE OVERLOADS

THEREFORE, THE SIDDHARTHS HAVE STUCK TO THIS SIMPLE EVENT FOR THEIR OBSERVATION :

A SIMPLE EVENT

THEY NOTICED THAT MADHAV ALWAYS STARTS CLIMBING THE STAIRS TO HIS ROOM AT STEP 0 AND EITHER CLIMBS ONE OR TWO STEPS AT A TIME WITH THE DECISION BEING RANDOM. HE NEVER DOES ANY OTHER ACTIVITY WHILE ON THE STAIRS

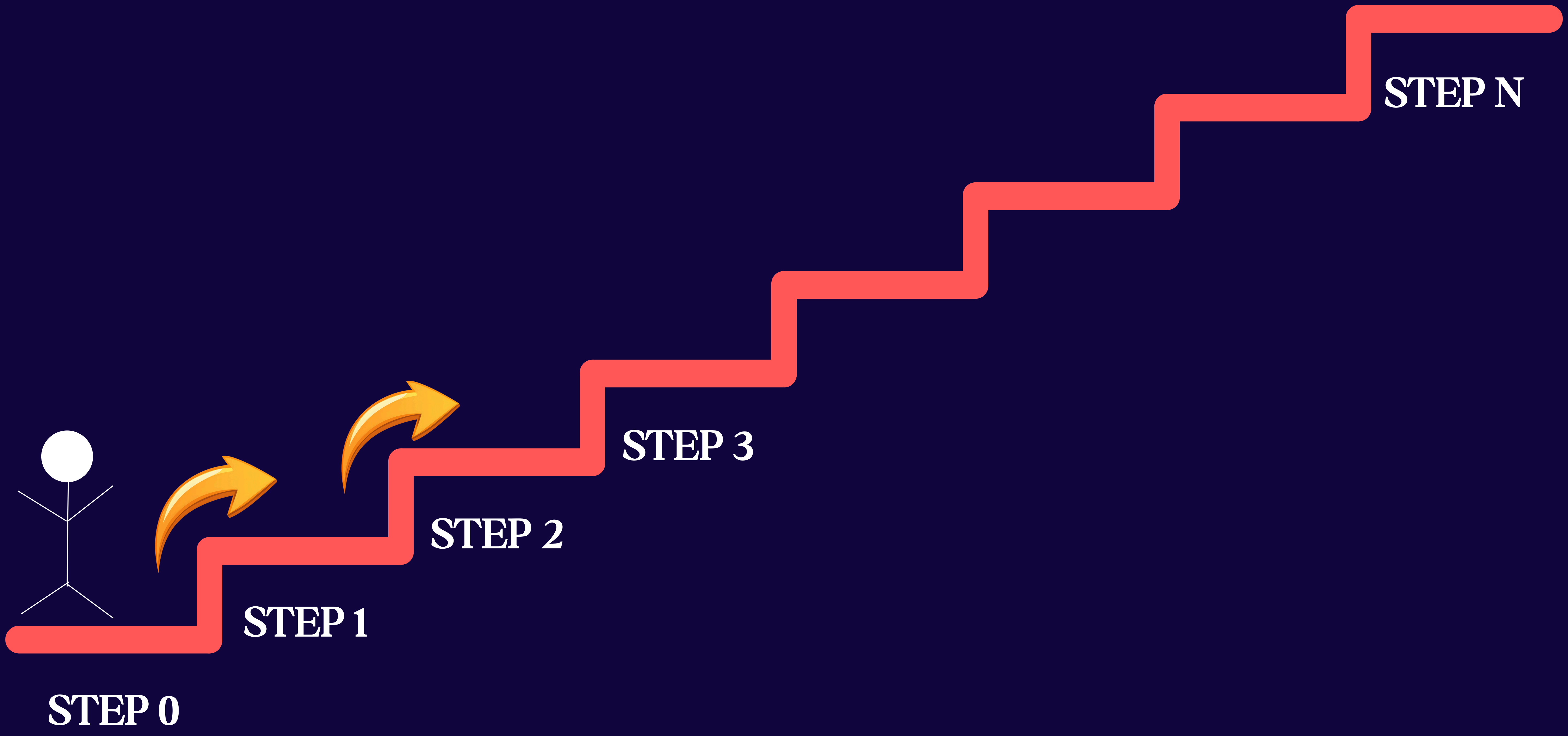
THE SIDDHARTHS BELIEVED THIS WAS A SIMPLE ENOUGH EVENT TO TEST THEIR MULTIVERSAL WINDOW ON

SO HERE’S THE QUESTION:

GIVEN THE TOTAL NUMBER OF STEPS, HOW MANY BRANCHES DO YOU THINK THE DEVICE WILL REGISTER

CONSIDER THAT THE DEVICE IS ACTIVATED WHEN MADHAV STARTS CLIMBING

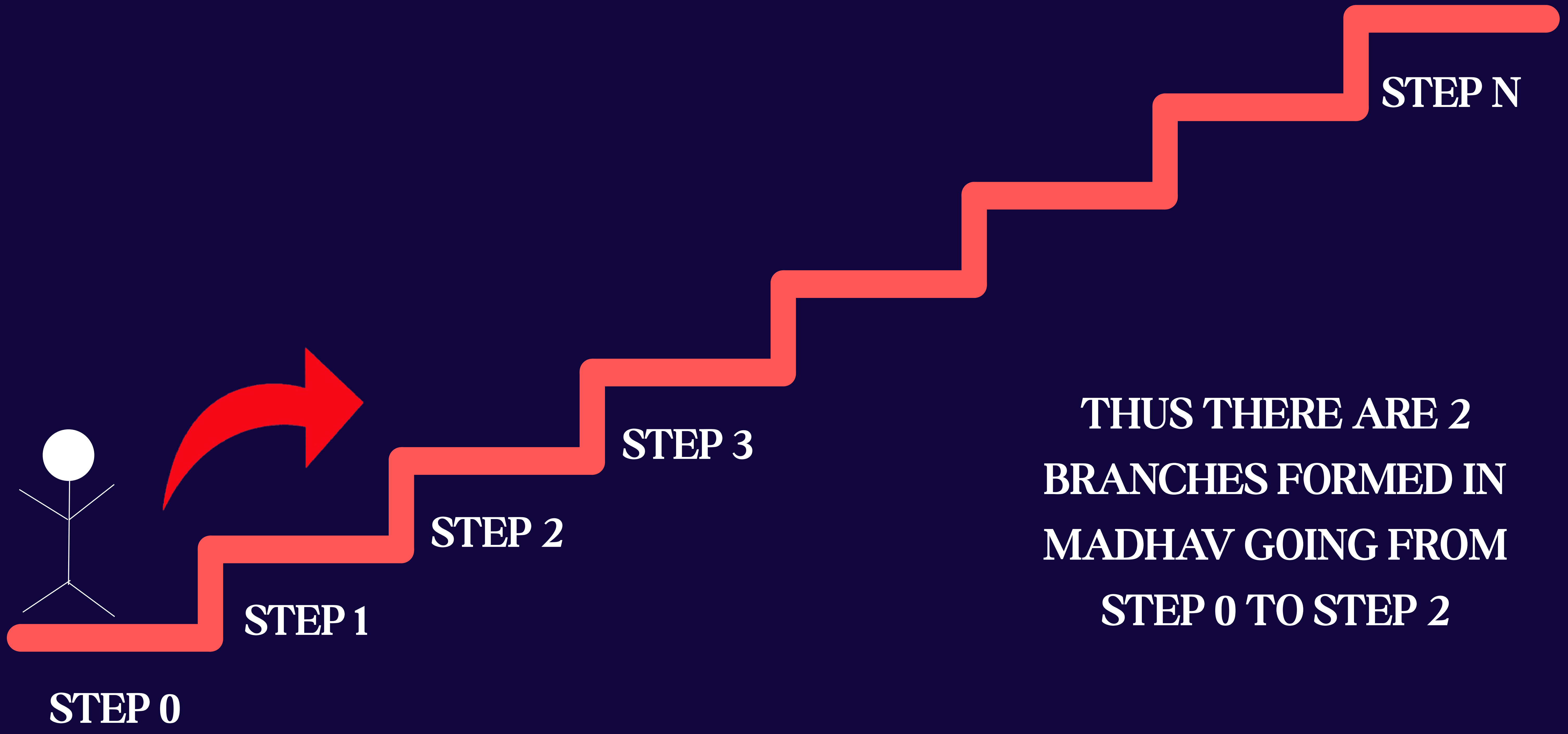
LET'S GO THROUGH A FEW STEPS MANUALLY.
CALCULATE THE NUMBER OF BRANCHES(NO. OF UNIQUE WAYS) IN REACHING STEP 2



ONE OF THE WAYS IS FOR HIM TO CLIMB ONE STEP AND THEN ANOTHER

LET'S GO THROUGH A FEW STEPS MANUALLY.

CALCULATE THE NUMBER OF BRANCHES(NO. OF UNIQUE WAYS) IN REACHING STEP 2



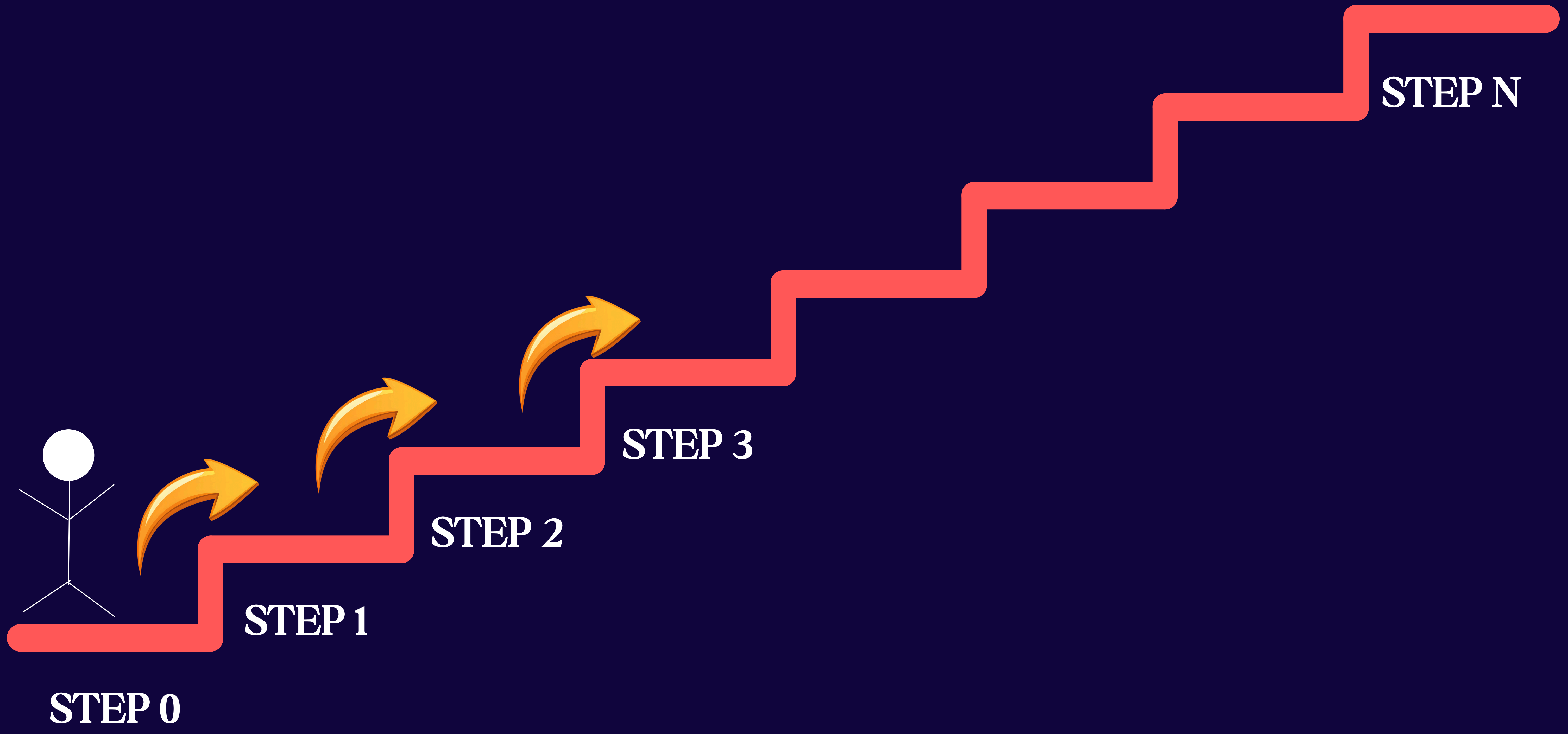
THUS THERE ARE 2 BRANCHES FORMED IN MADHAV GOING FROM STEP 0 TO STEP 2

OR HE COULD DIRECTLY JUMP 2 STEPS AND REACH IT IN ONE MOVE

LET'S GO THROUGH A FEW STEPS MANUALLY.

NOW, LET'S CALCULATE THE NUMBER OF BRANCHES(NO. OF UNIQUE WAYS) IN REACHING

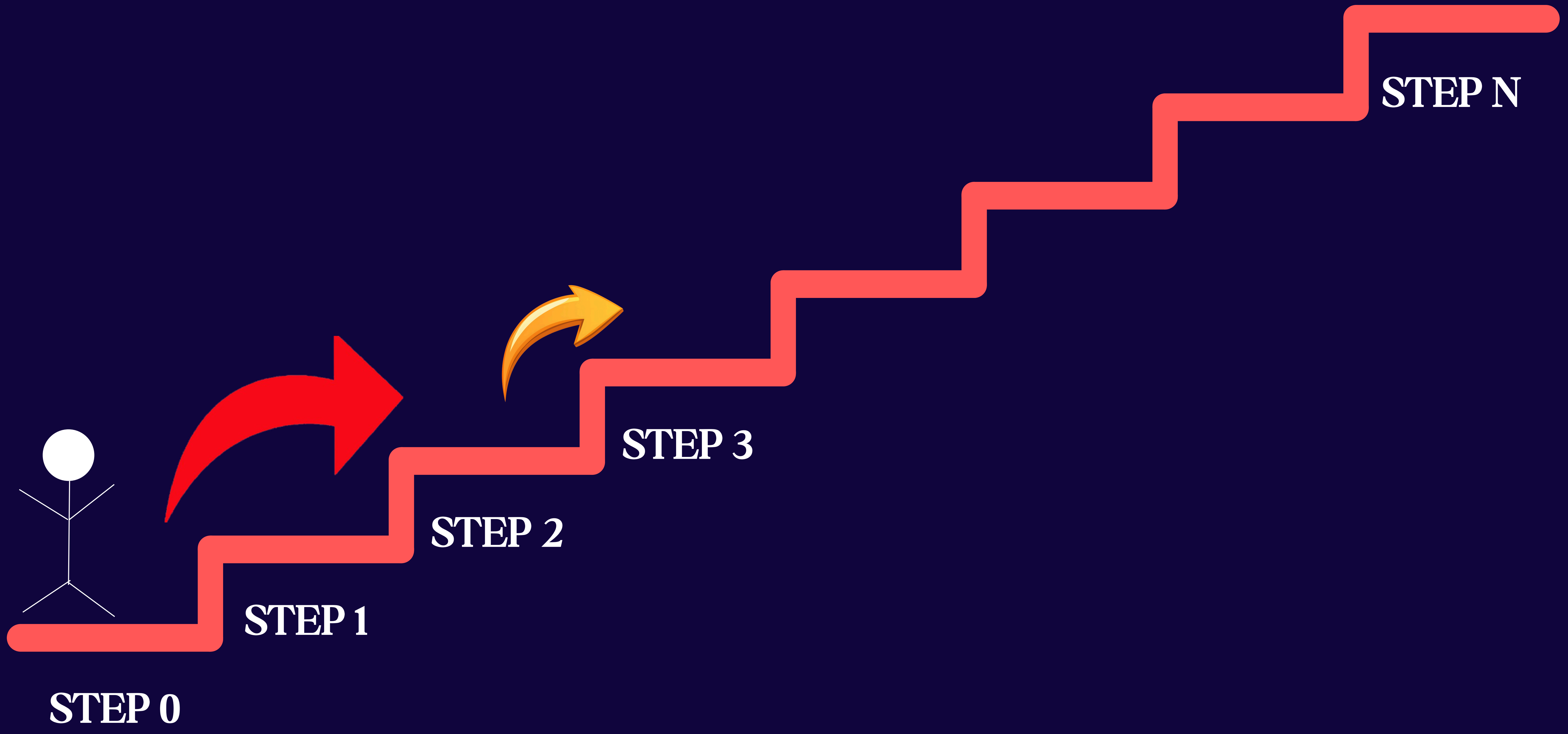
STEP 3



AGAIN HE CAN GO ONE STEP AT A TIME AND REACH STEP 3

LET'S GO THROUGH A FEW STEPS MANUALLY.

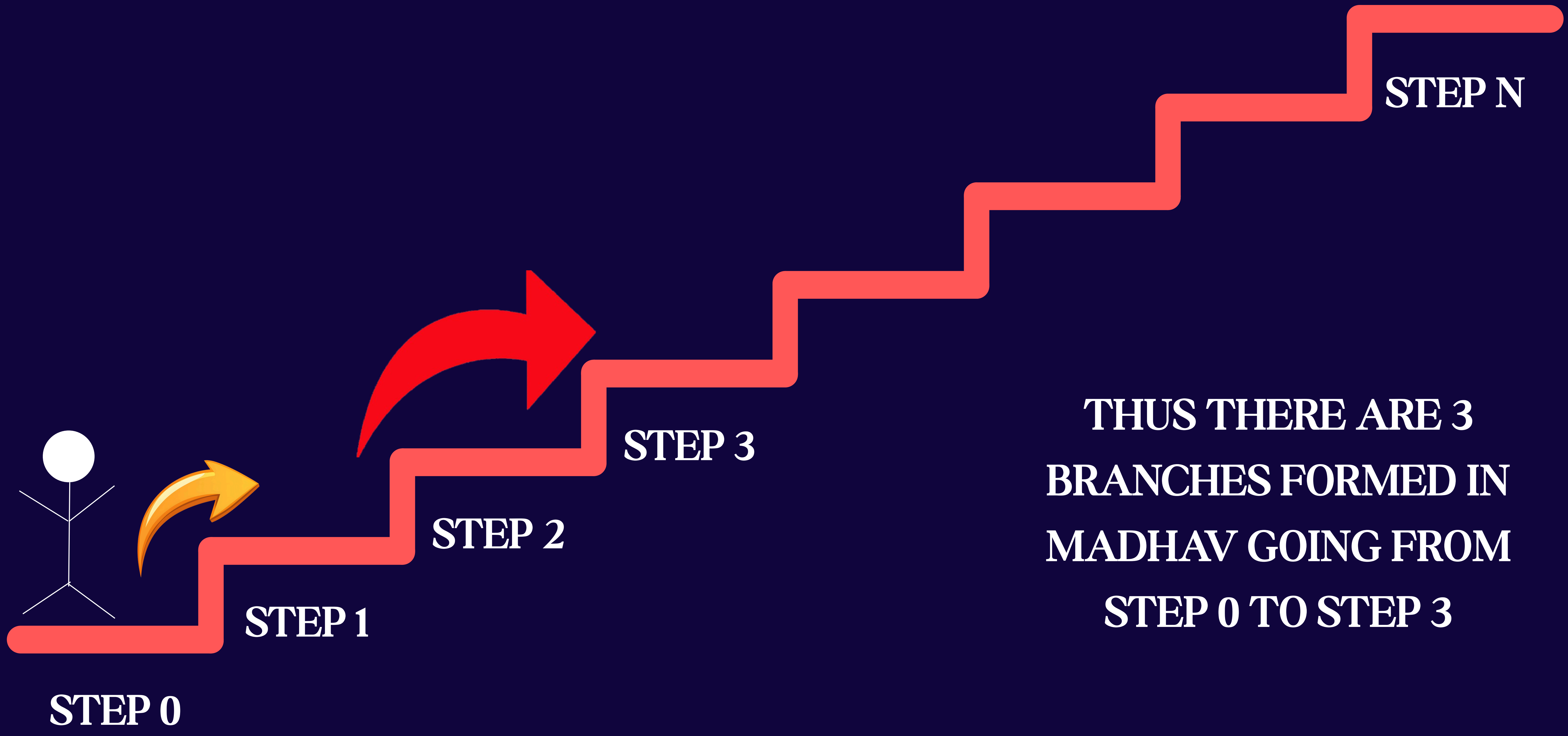
NOW, LET'S CALCULATE THE NUMBER OF BRANCHES(NO. OF UNIQUE WAYS) IN REACHING *STEP 3*



OR HE CAN JUMP TO STEP 2 AND THEN TAKE ONE STEP

LET’S GO THROUGH A FEW STEPS MANUALLY.

NOW, LET’S CALCULATE THE NUMBER OF BRANCHES(NO. OF UNIQUE WAYS) IN REACHING *STEP 3*

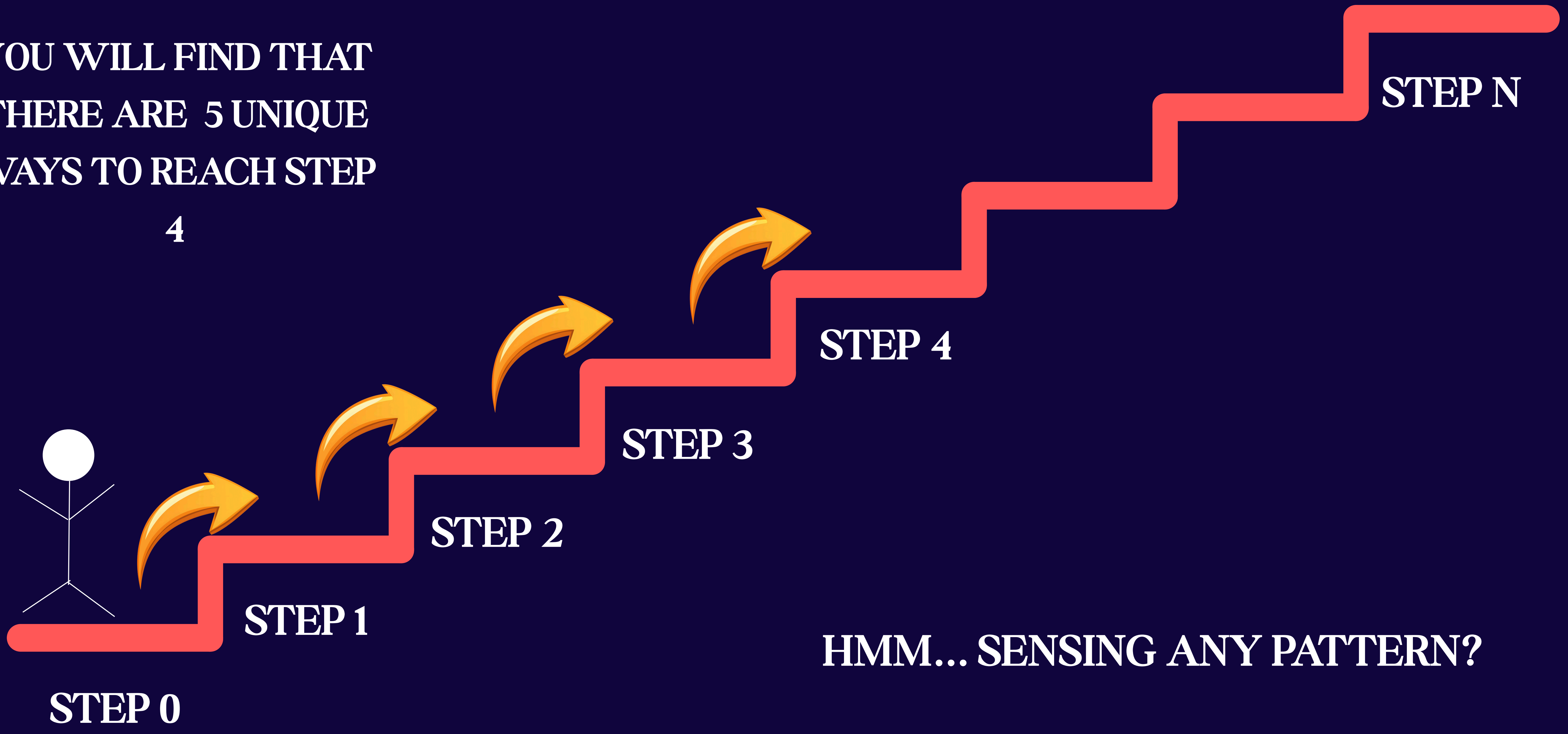


THUS THERE ARE 3 BRANCHES FORMED IN MADHAV GOING FROM STEP 0 TO STEP 3

OR TAKE A STEP TO STEP 1 AND THEN JUMP TO STEP 3

NOW, TO CALCULATE FOR STEP 4, YOU CAN REPEAT THE SAME ANALYSIS

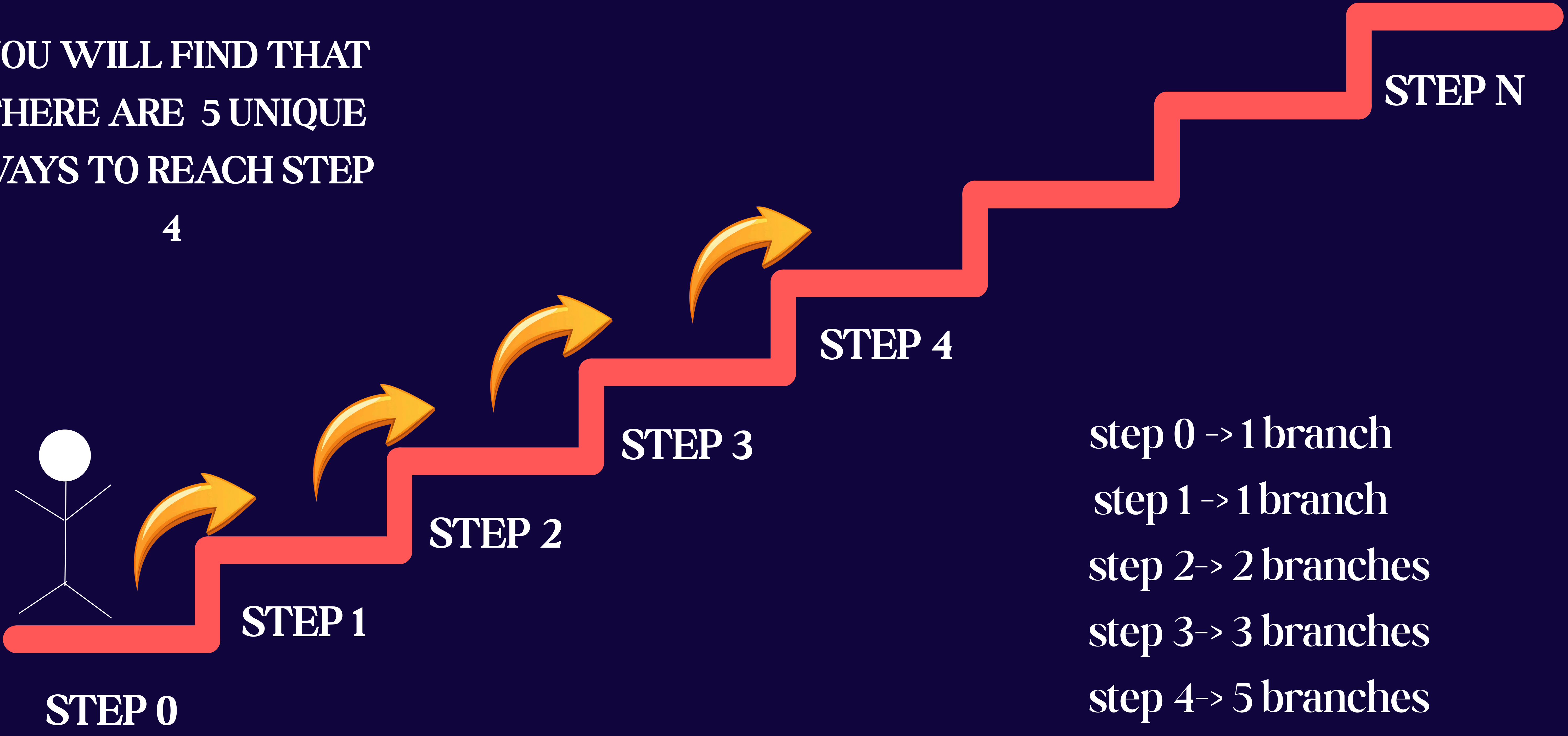
YOU WILL FIND THAT THERE ARE 5 UNIQUE WAYS TO REACH STEP 4



HMM... SENSING ANY PATTERN?

NOW, TO CALCULATE FOR STEP 4, YOU CAN REPEAT THE SAME ANALYSIS

YOU WILL FIND THAT THERE ARE 5 UNIQUE WAYS TO REACH STEP 4



step 0 -> 1 branch

step 1 -> 1 branch

step 2-> 2 branches

step 3-> 3 branches

step 4-> 5 branches

GETTING IT NOW?

YEP! YOU’RE RIGHT ! THE TREND IT FOLLOWS IS INDEED THE FIBONACCI SEQUENCE.
0 1 1 2 3 5 8 13 21 AND SO ON..

NOW, COME UP WITH A MATHEMATICAL EQN TO REPRESENT ANY GENERAL FIBONACCI NUMBER i

YEP! YOU'RE RIGHT ! THE TREND IT FOLLOWS IS INDEED THE FIBONACCI SEQUENCE.
0 1 1 2 3 5 8 13 21 AND SO ON..

NOW, COME UP WITH A MATHEMATICAL EQN TO REPRESENT ANY GENERAL FIBONACCI NUMBER i

$$f(i) = f(i-1) + f(i-2)$$

Now, does this make sense in our case?
In our question, f(i) would give the number of branches possible.

So, this equation would mean that
No. of ways to reach step i = no. of ways to reach (i -1) + no. of ways to reach (i-2)

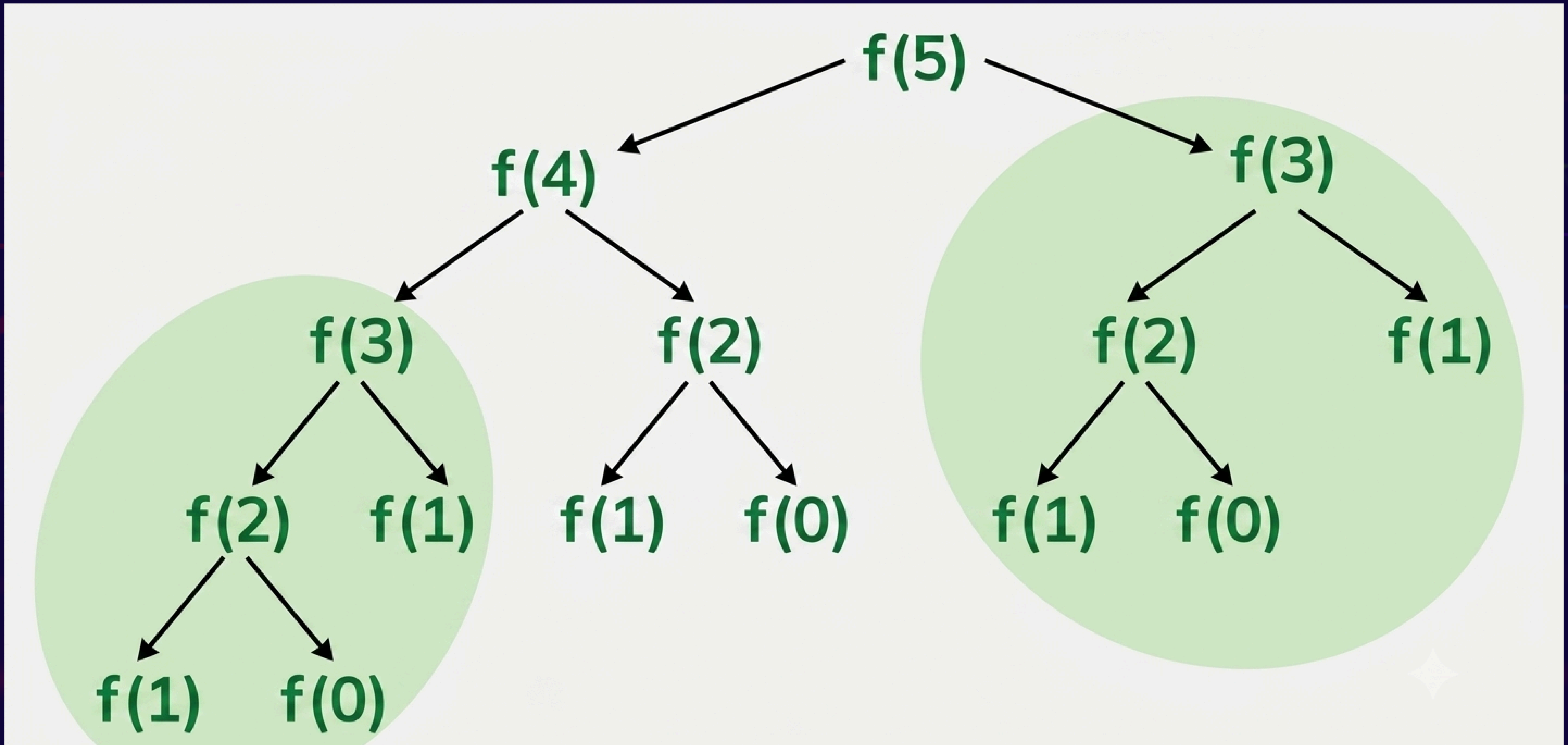
$f(i) = f(i-1) + f(i-2)$

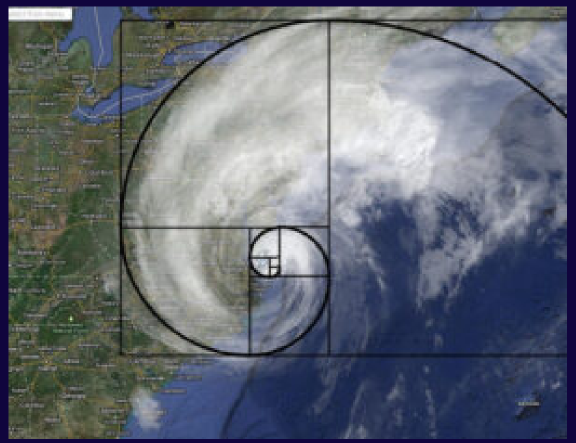
Where f(i) represents the ith fibonacci number

A function which calls upon the same function with smaller and simpler arguments breaking the problem down into subproblems is called a

“RECURSIVE FUNCTION“

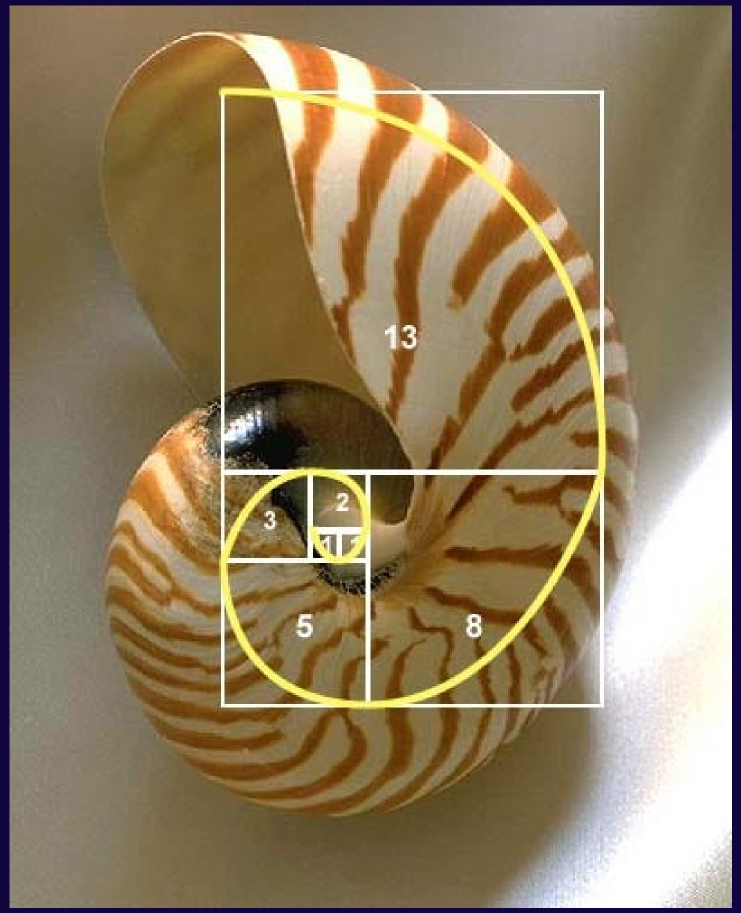
AND SUCH A PROCESS IS CALLED **RECURSION**

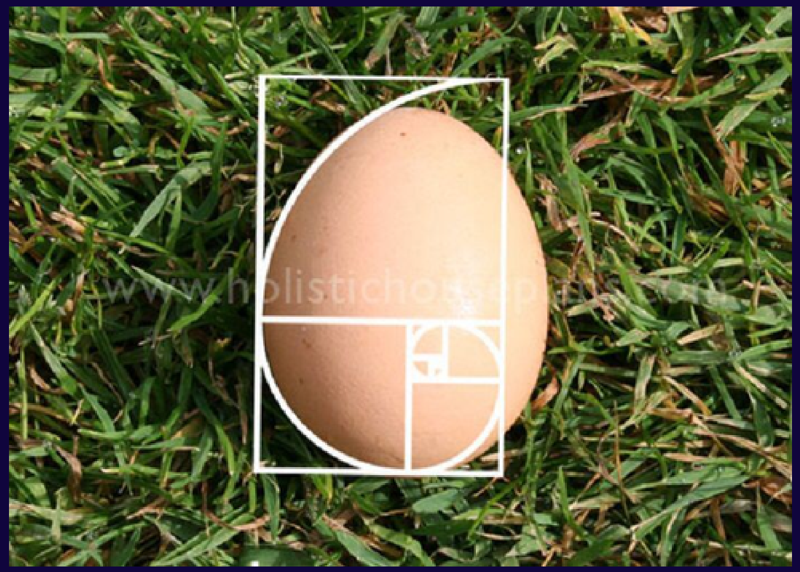




calculating fibonacci numbers!

now the real fun starts :]





a glimpse into the future

we’ll go into 3 methods to calculate the nth fibonacci number

a sneak peak into their performances:
(measured in nth fibonacci number in 1 ms*)

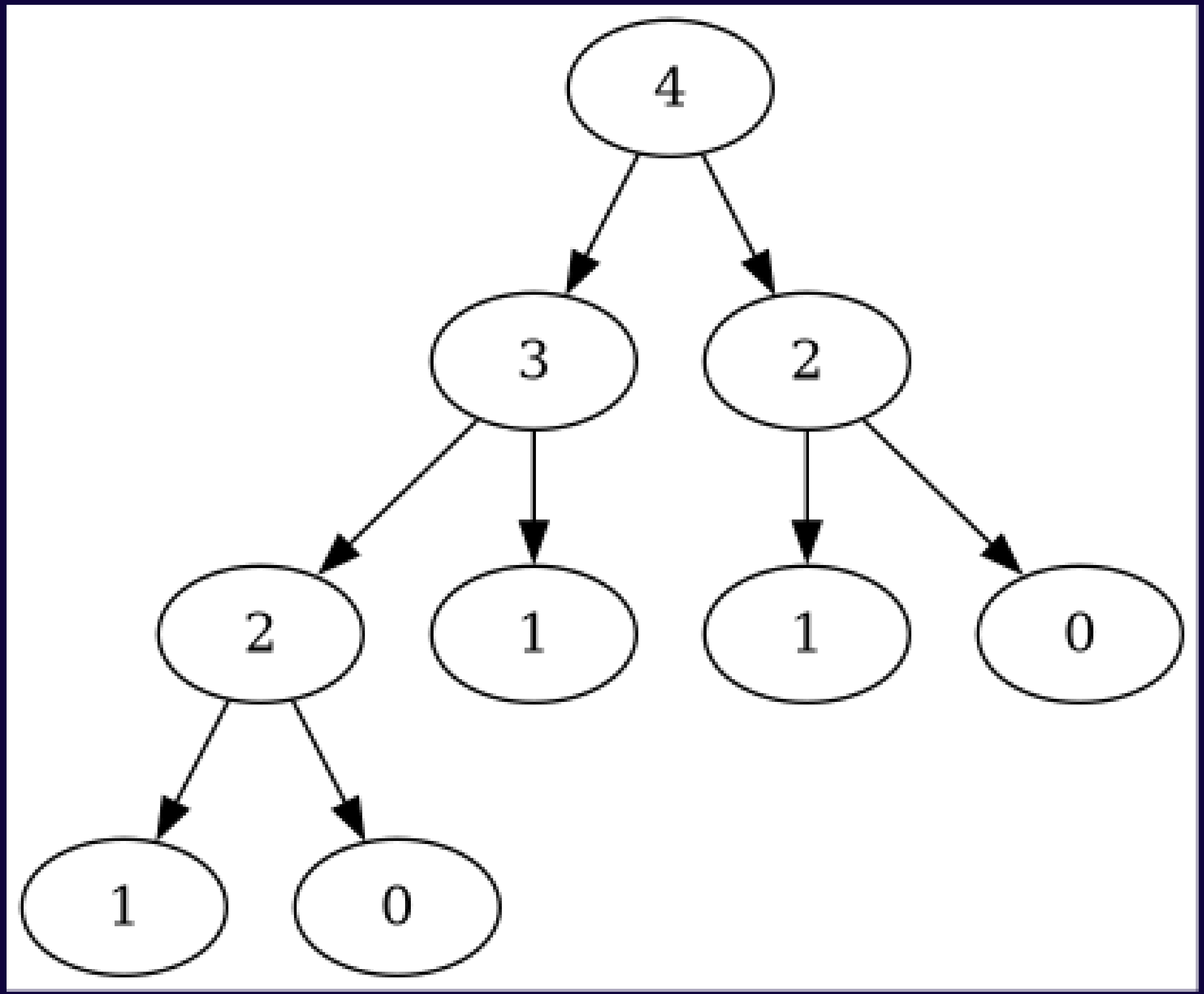
method 1 → n = 26

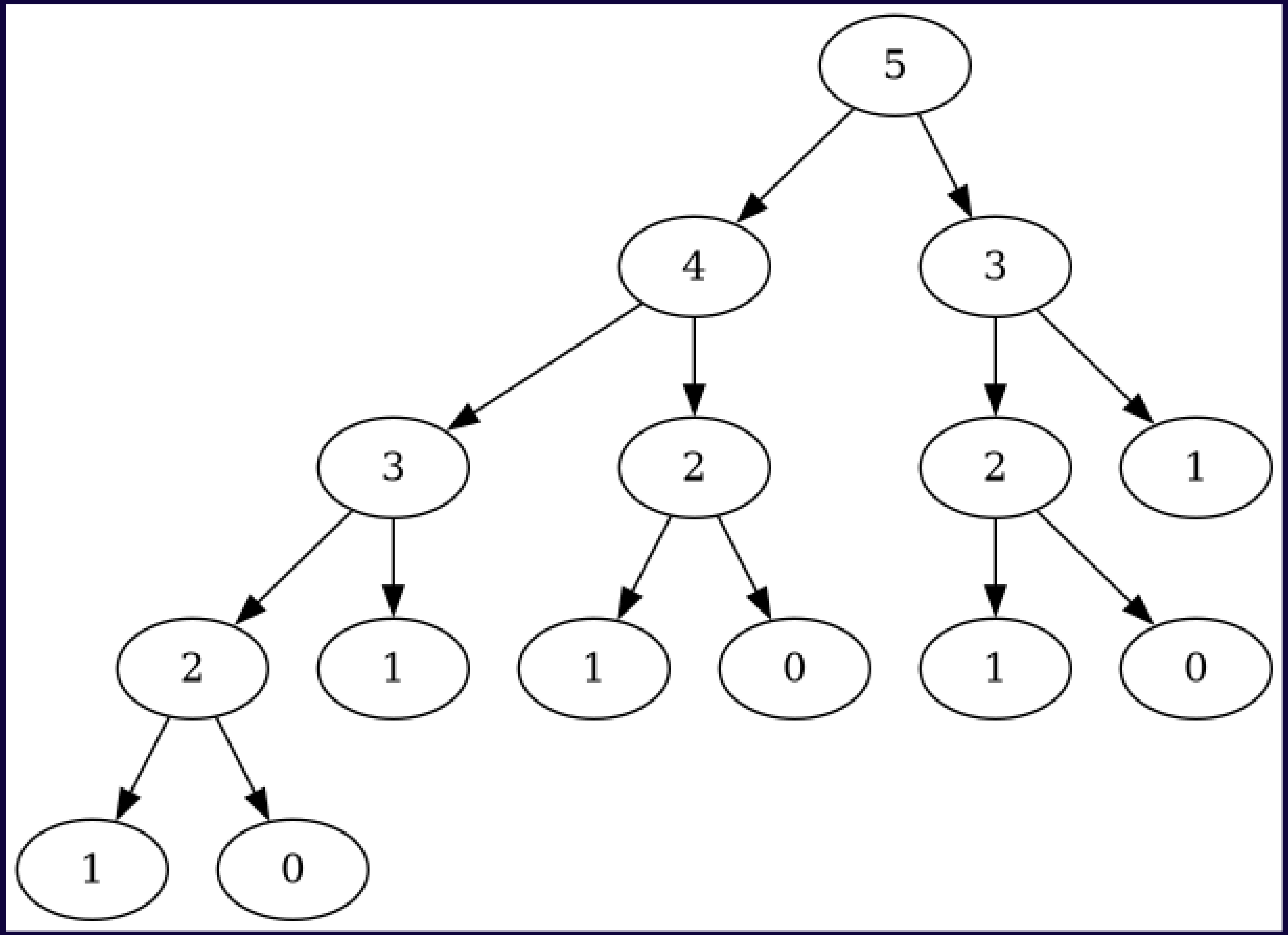
method 2 → n = around 23k

method 3 → n = around 2^{100k} = around 10^{2900}

*computers nowadays are so fast that in this case we’re measuring in the scale of a millisecond, of course that creates problems of its own......

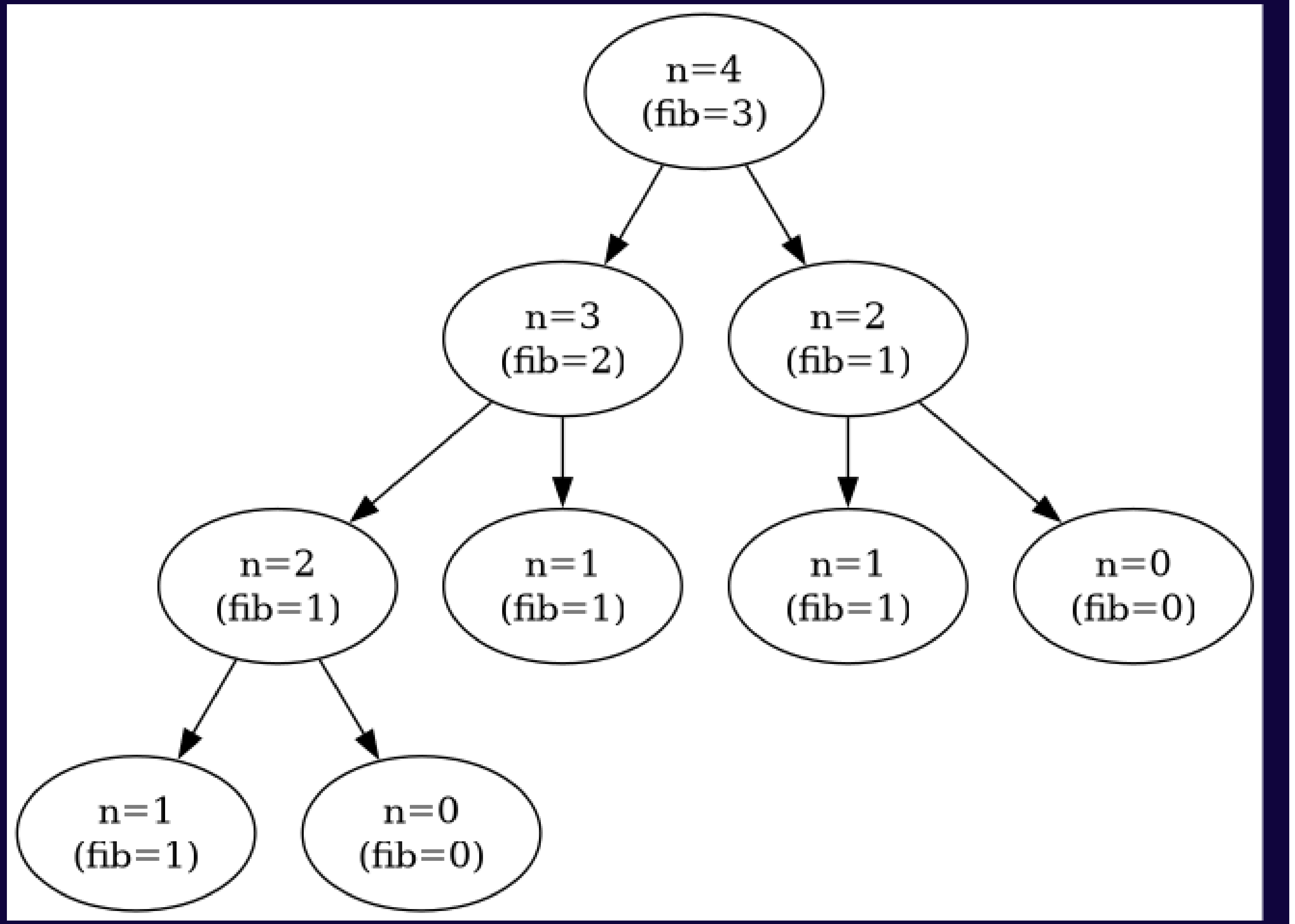
calculating recursively (from the top)

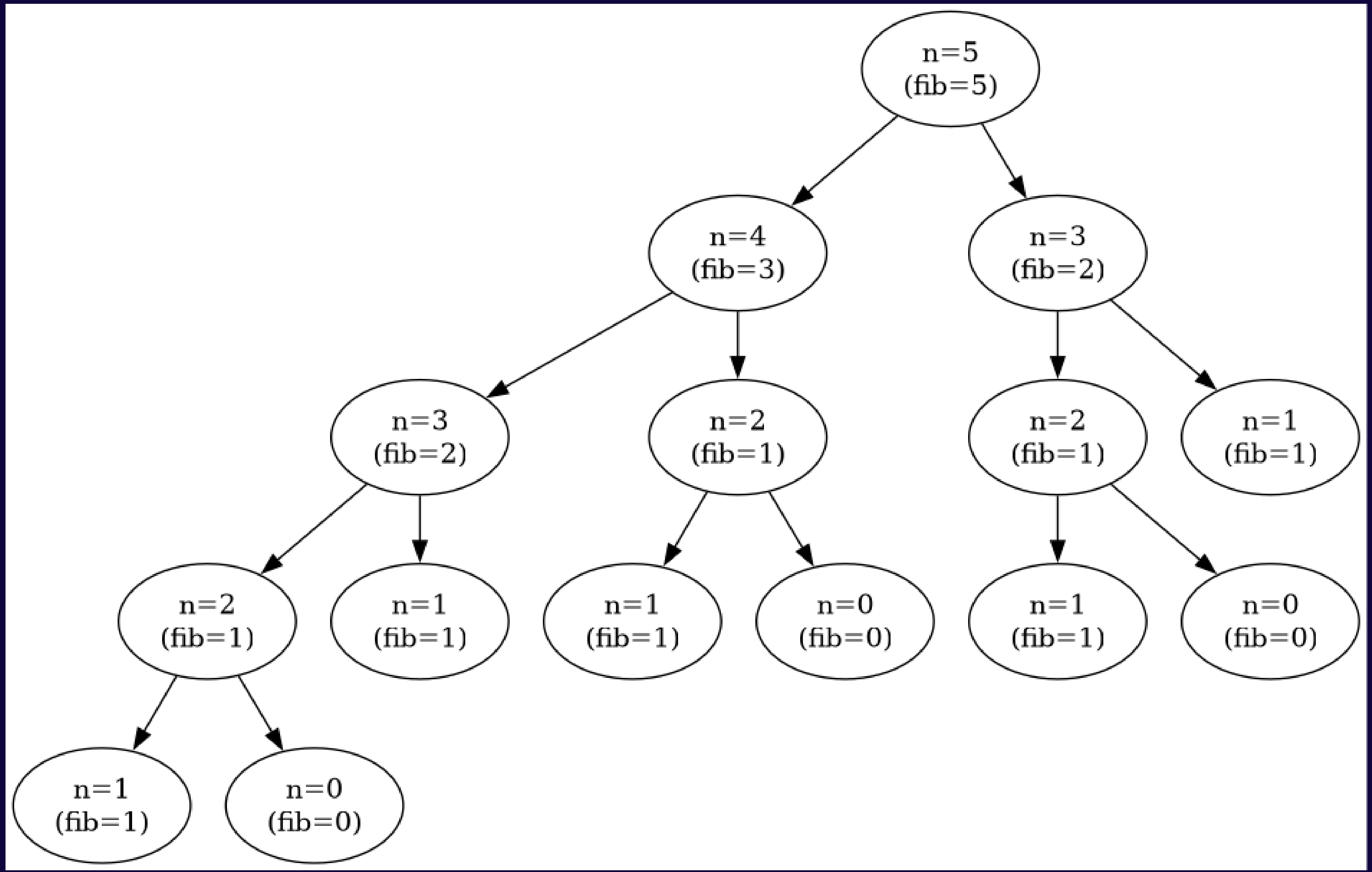




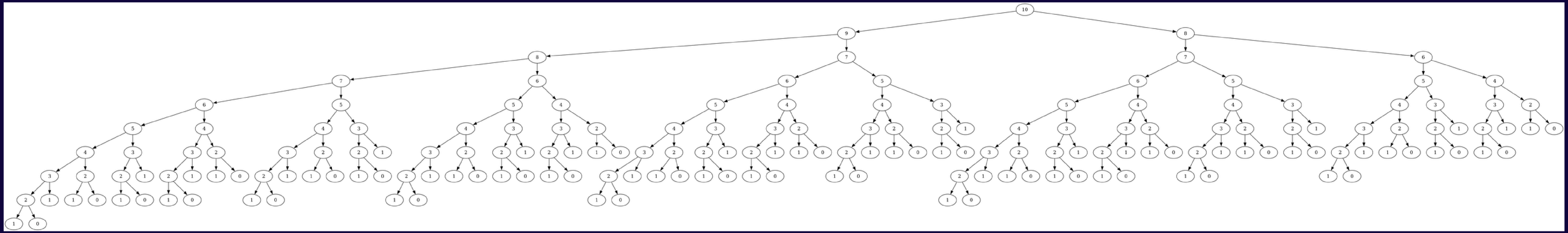
calculating recursively (from the top)

values flow up





the problem?



n = 10 => 177 nodes

n = 15 => 1973 nodes

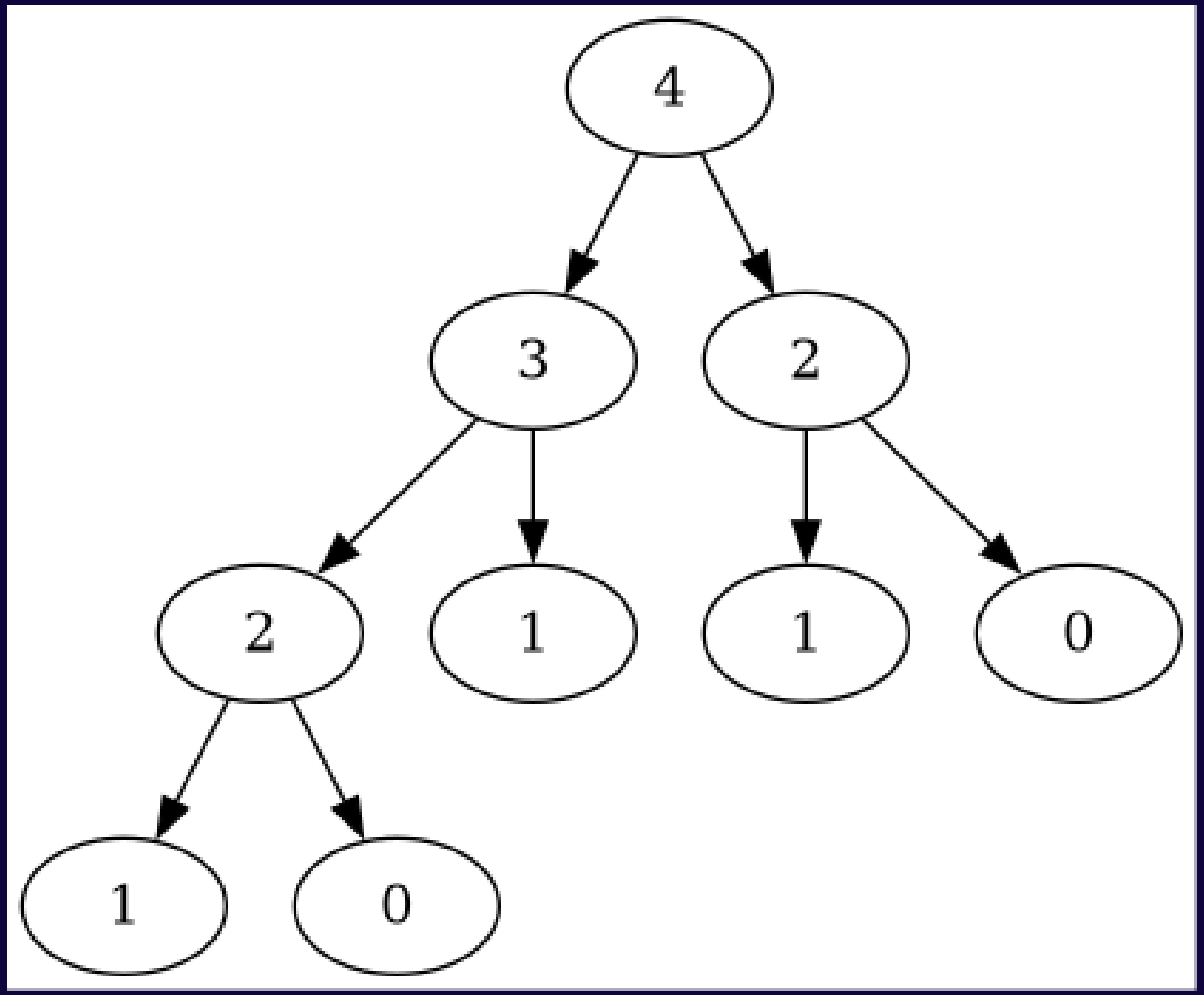
n = 50 => 40,730,022,198 nodes

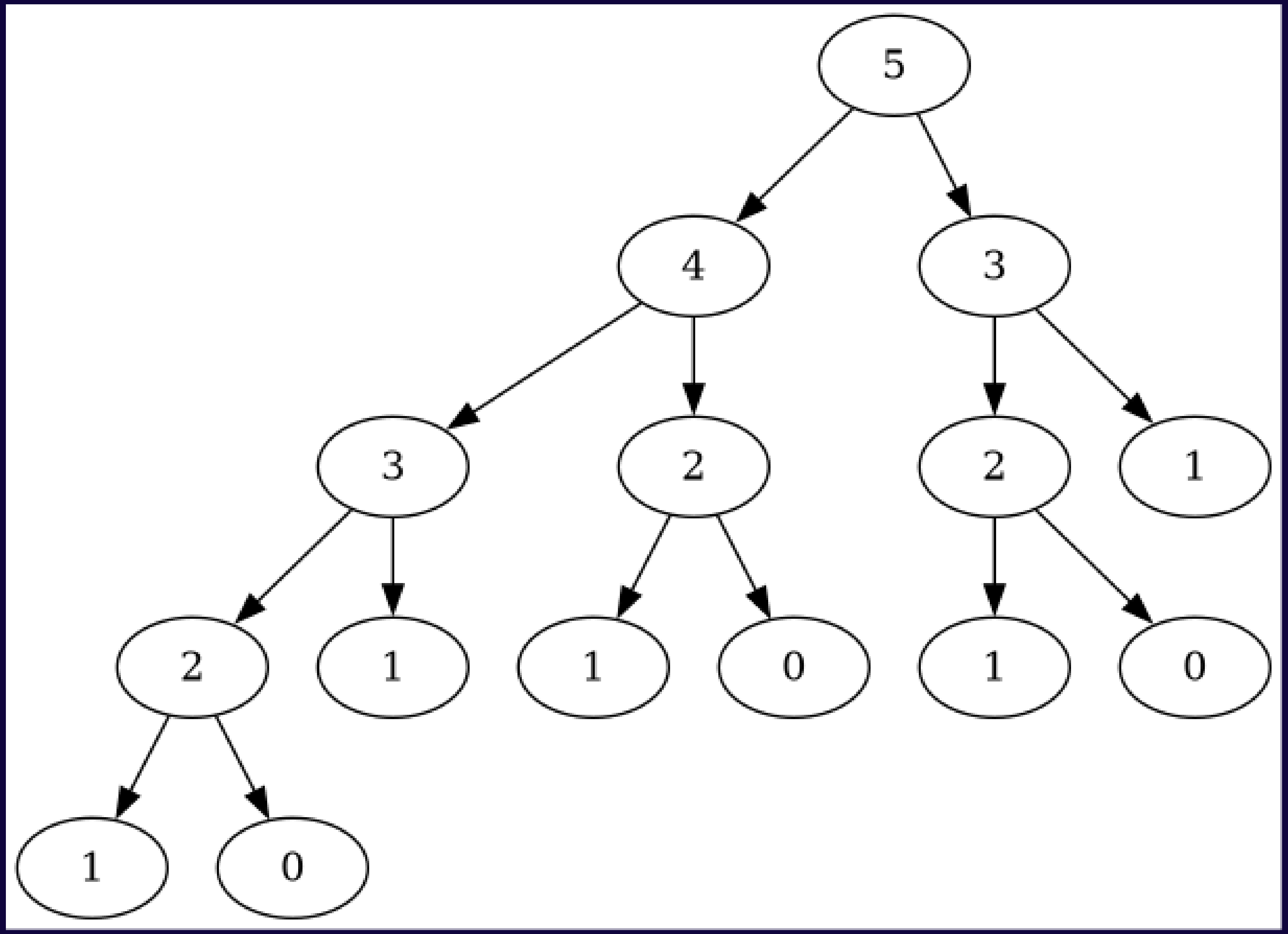
....

n = 1000 => 10^{209} nodes

back to the start

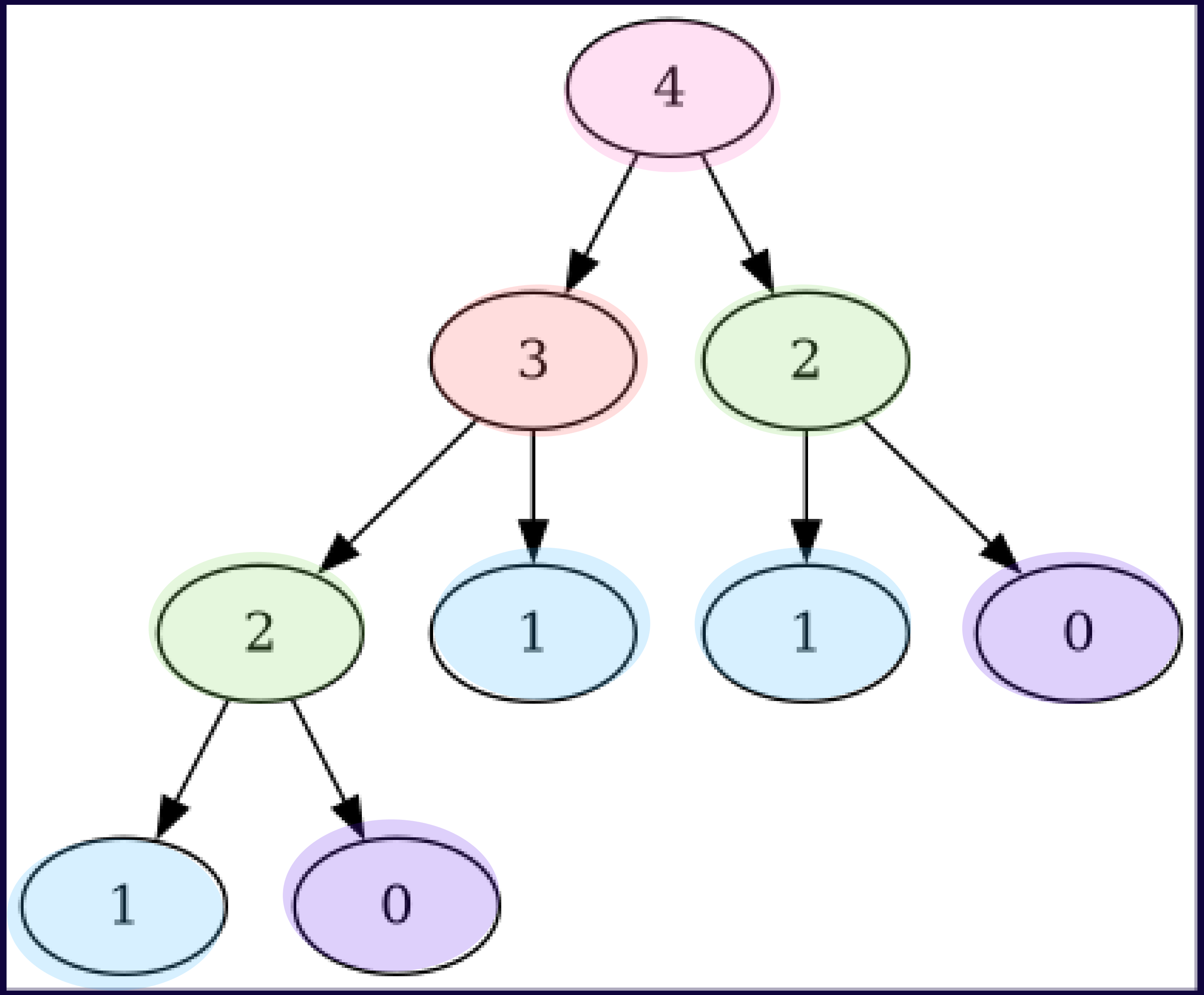
hmmmm, do you notice something?

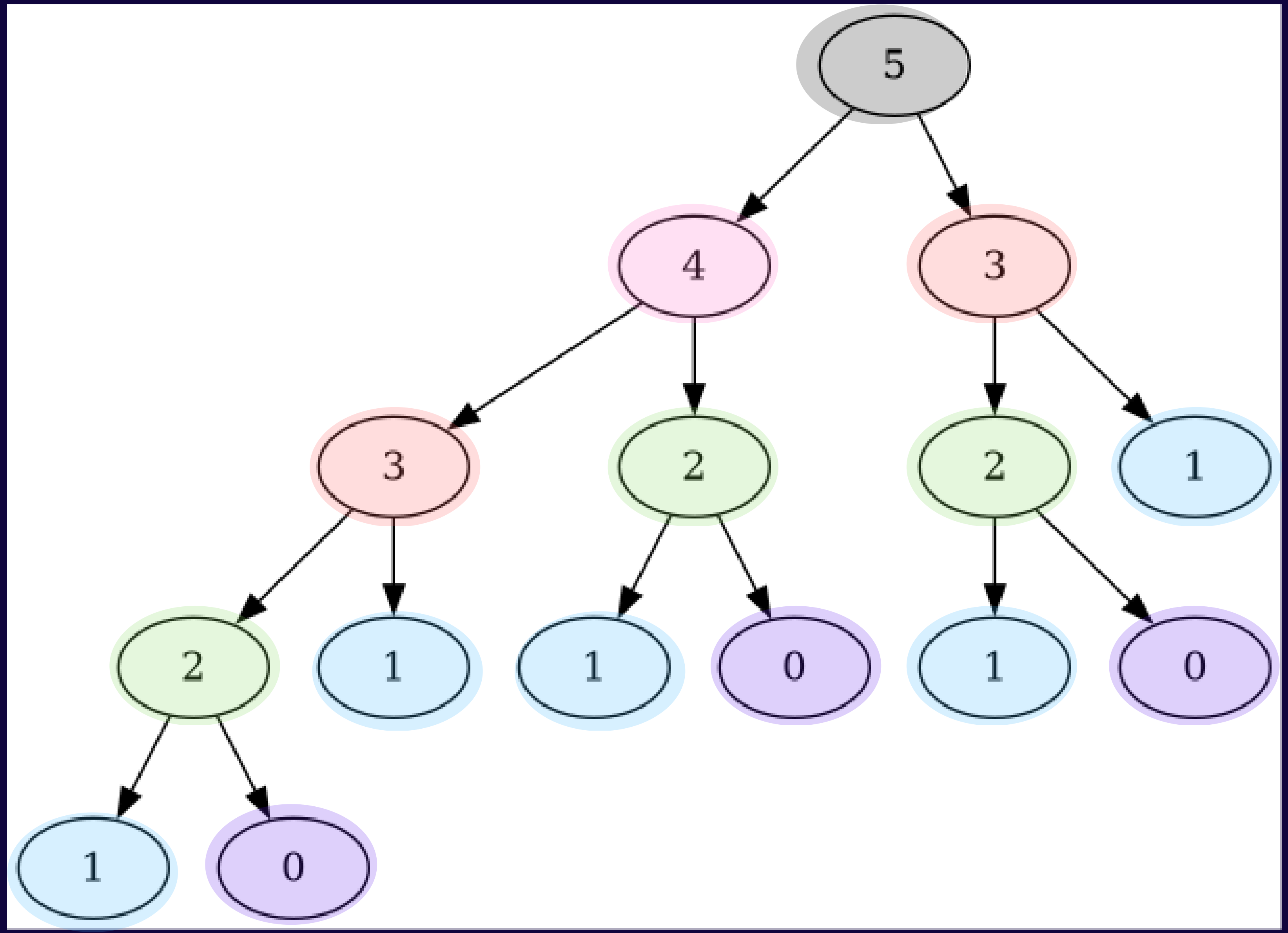




back to the start

hmmmm, do you notice something?





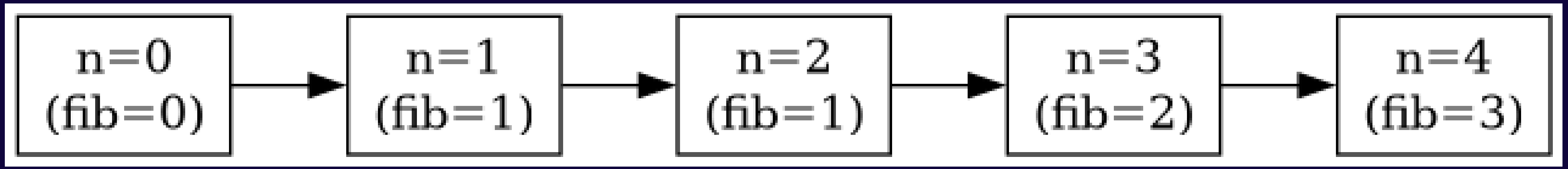
the same number is calculated again and again!

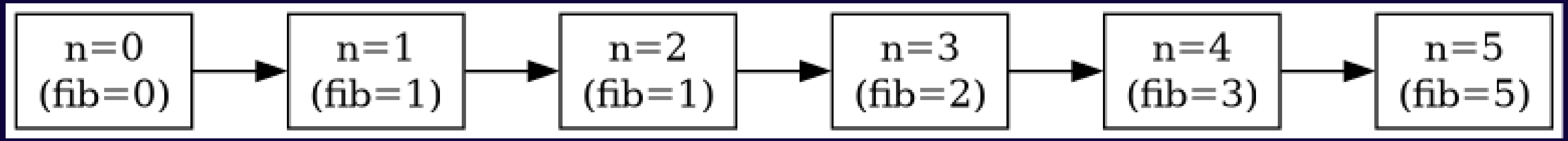
reversing directions
let's go from the bottom this time

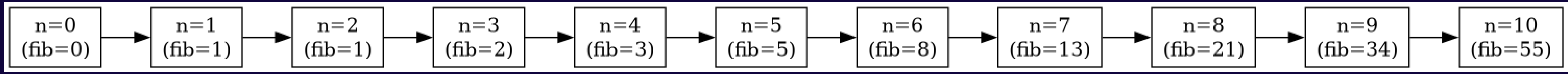
fib(0) = 0
fib(1) = 1
fib(2) = 1
fib(3) = 2
fib(4) = 3
fib(5) = 5
fib(6) = 8
fib(7) = 13
......

values flow down

linear chains(iterating from the bottom)







only 11 nodes now, not 177!

any doubts till now?

CAN WE DO BETTER?

introducing.... matrices?

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

the humble matrix

$$\begin{bmatrix} 5 \\ 6 \end{bmatrix}$$

just a vector

$$\begin{bmatrix} \color{red}{\mathbf{1}} & \color{red}{\mathbf{2}} \\ 3 & 4 \end{bmatrix} \begin{bmatrix} \color{blue}{\mathbf{5}} & 6 \\ \color{blue}{\mathbf{7}} & 8 \end{bmatrix} = \begin{bmatrix} (\color{red}{\mathbf{1}} \cdot \color{blue}{\mathbf{5}} + \color{red}{\mathbf{2}} \cdot \color{blue}{\mathbf{7}}) & (1 \cdot 6 + 2 \cdot 8) \\ (3 \cdot 5 + 4 \cdot 7) & (3 \cdot 6 + 4 \cdot 8) \end{bmatrix} = \begin{bmatrix} \mathbf{19} & 22 \\ 43 & 50 \end{bmatrix}$$

a matrix-matrix product

$$\begin{bmatrix} \color{red}{\mathbf{1}} & \color{red}{\mathbf{2}} \\ 3 & 4 \end{bmatrix} \begin{bmatrix} \color{blue}{\mathbf{5}} \\ \color{blue}{\mathbf{6}} \end{bmatrix} = \begin{bmatrix} (\color{red}{\mathbf{1}} \cdot \color{blue}{\mathbf{5}} + \color{red}{\mathbf{2}} \cdot \color{blue}{\mathbf{6}}) \\ (3 \cdot 5 + 4 \cdot 6) \end{bmatrix} = \begin{bmatrix} \mathbf{17} \\ 39 \end{bmatrix}$$

a matrix-vector product

now, magic!

$$\begin{bmatrix} ? & ? \\ ? & ? \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

can you guess the entries?

voila!

$$\begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} (1 \cdot 1 + 1 \cdot 0) \\ (1 \cdot 1 + 0 \cdot 0) \end{bmatrix} = \begin{bmatrix} \mathbf{1} \\ 1 \end{bmatrix}$$

we have the fibonacci matrix!

a few generations, for good measure

$$\begin{bmatrix}1 & 1\\1 & 0\end{bmatrix}\begin{bmatrix}1\\1\end{bmatrix} = \begin{bmatrix}1\cdot 1 + 1\cdot 1\\1\cdot 1 + 0\cdot 1\end{bmatrix} = \begin{bmatrix}\mathbf{2}\\1\end{bmatrix}$$

$$\begin{bmatrix}1 & 1\\1 & 0\end{bmatrix}\begin{bmatrix}2\\1\end{bmatrix} = \begin{bmatrix}1\cdot 2 + 1\cdot 1\\1\cdot 2 + 0\cdot 1\end{bmatrix} = \begin{bmatrix}\mathbf{3}\\2\end{bmatrix}$$

$$\begin{bmatrix}1 & 1\\1 & 0\end{bmatrix}\begin{bmatrix}3\\2\end{bmatrix} = \begin{bmatrix}1\cdot 3 + 1\cdot 2\\1\cdot 3 + 0\cdot 2\end{bmatrix} = \begin{bmatrix}\mathbf{5}\\3\end{bmatrix}$$

but why?

formally,

$$\begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^n \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} F_{n+1} \\ F_n \end{bmatrix}$$

$$\underbrace{\begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix} \cdots \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}}_{n \text{ times}} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} F_{n+1} \\ F_n \end{bmatrix}$$

isn’t ‘n’ number of steps, the same thing as before?
how will we get any better?

digression - binary numbers

in the base 10/decimal system, we have $123 = 1*10^2 + 2*10^1 + 3*10^0$

same thing in base 2, $123 = 64 + 32 + 16 + 8 + 2 + 1$
$= 1*2^6 + 1*2^5 + 1*2^4 + 1*2^3 + 0*2^2 + 1*2^1 + 1*2^0$
so in binary representation 123 is 1111011

in general, a positive integer ‘n’ needs $\rfloor \log_2(n) \lfloor +1$ bits

the exponent trick

let’s try calculating, let’s say, 67^n first

for n = 10, $67^{10} = 67*67*67*...*67$ → that’s 10 multiplications in total

however, let’s write 10 in binary: 10 = 8 + 2 => 1010 in binary representation

we can calculate, 67^2, 67^4 and 67^8 in advance → that’s 3 multiplications
and $67^{10} = 67^{(8+2)} = 67^8 * 67^2$ → that’s 1 multiplication,
for a total of 4 multiplications

can you figure out the number of multiplications required for a general ‘n’?

the exponent trick

for 67^n, let's say n needs k+1 bits to represent it in binary
(here $k = \lfloor \log_2(n) \rfloor$)

we calculate, 67^2, 67^4, ..., 67^{2^k} → that's k multiplications

similar to before, whichever powers of 2 are involved in the binary representation of n, are also involved in the actual product
from this, we have (number of '1's in binary representation - 1) multiplications

for example, $10 \equiv 1010$, has 2 '1's in its binary representation
=> 1 multiplication required

the exact number of multiplications is cool, but an upper bound would be cooler!

spoilers: logarithmic upper bound

number of 1s in the binary representation <= k+1 (number of bits)

total number of multiplications <= k + (k+1) - 1 = 2*k = $2 * \rfloor \log_2(n) \lfloor$

upper bound on multiplications = $2 * \rfloor \log_2(n) \lfloor$

this is way, way better than the n multiplications earlier for large n back to matrices

$$\begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^n \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} F_{n+1} \\ F_n \end{bmatrix}$$

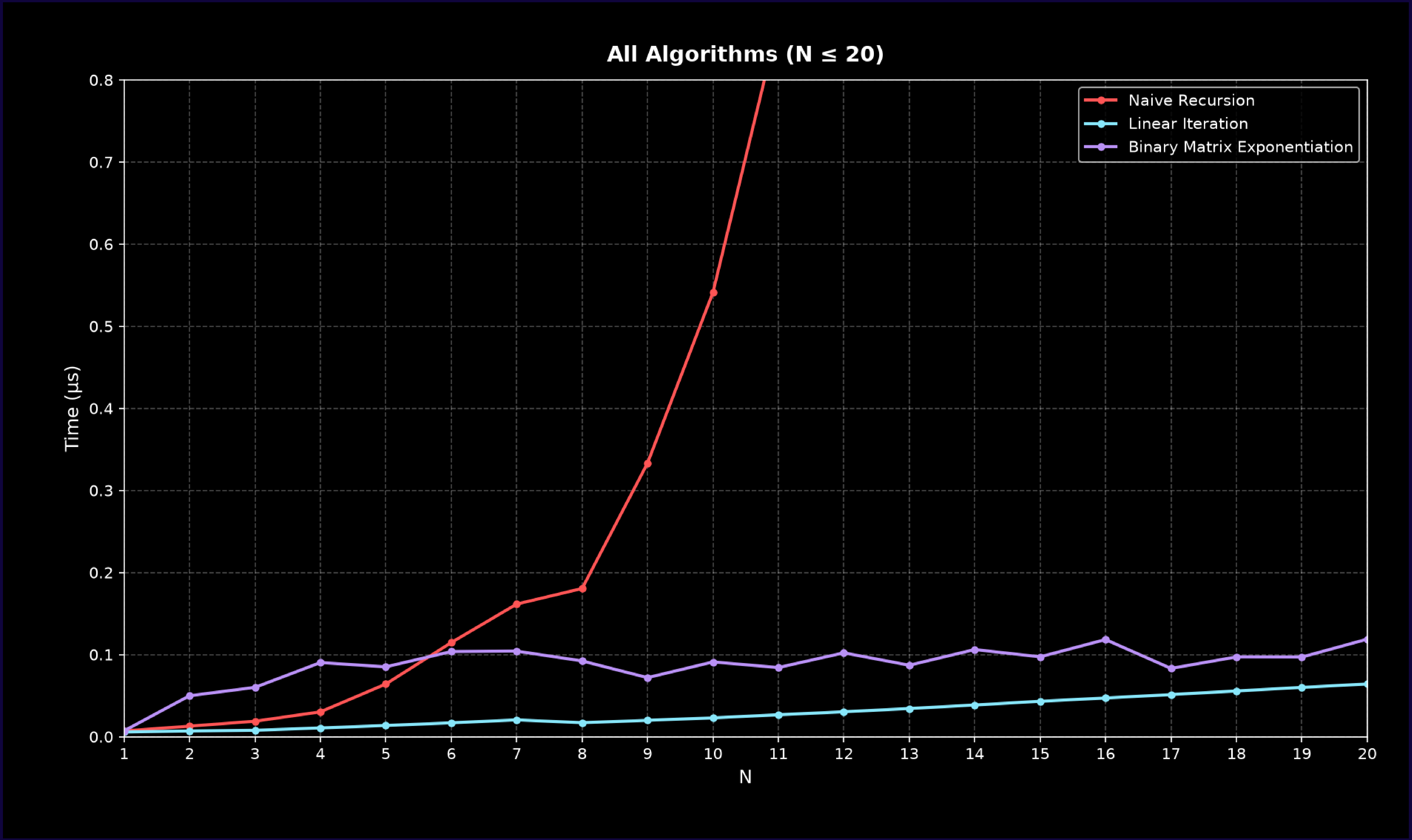
let's look at n = 13, as an example

Binary Expansion: $13 = \mathbf{8} + \mathbf{4} + \mathbf{1} = \mathbf{2^3} + \mathbf{2^2} + \mathbf{2^0}$

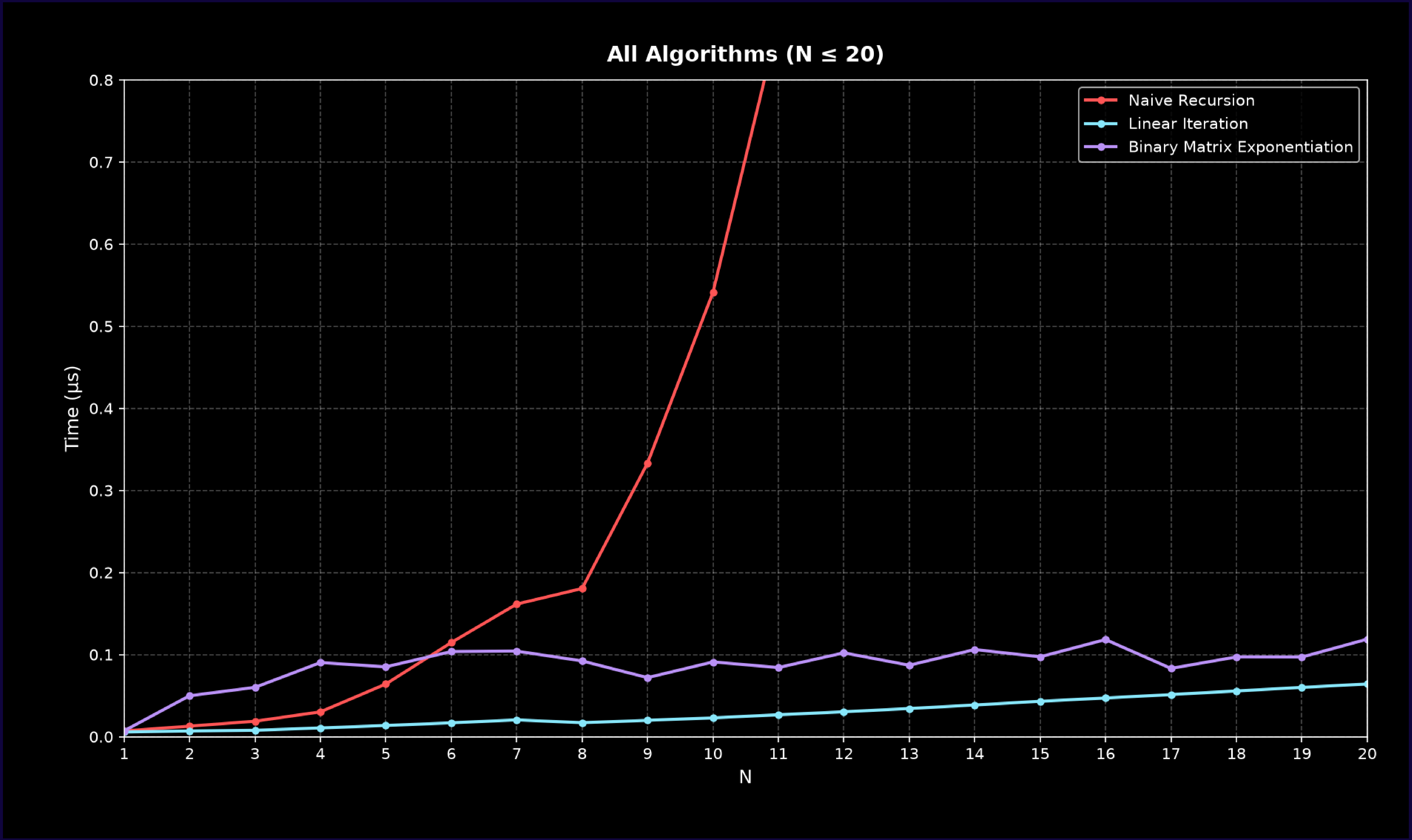
$$\begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^{\mathbf{13}} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^{\mathbf{2^3+2^2+2^0}} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^{\mathbf{8}} \times \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^{\mathbf{4}} \times \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^{\mathbf{1}}$$

this is the exact same concept, just replace multiplications with matrix multiplications :P

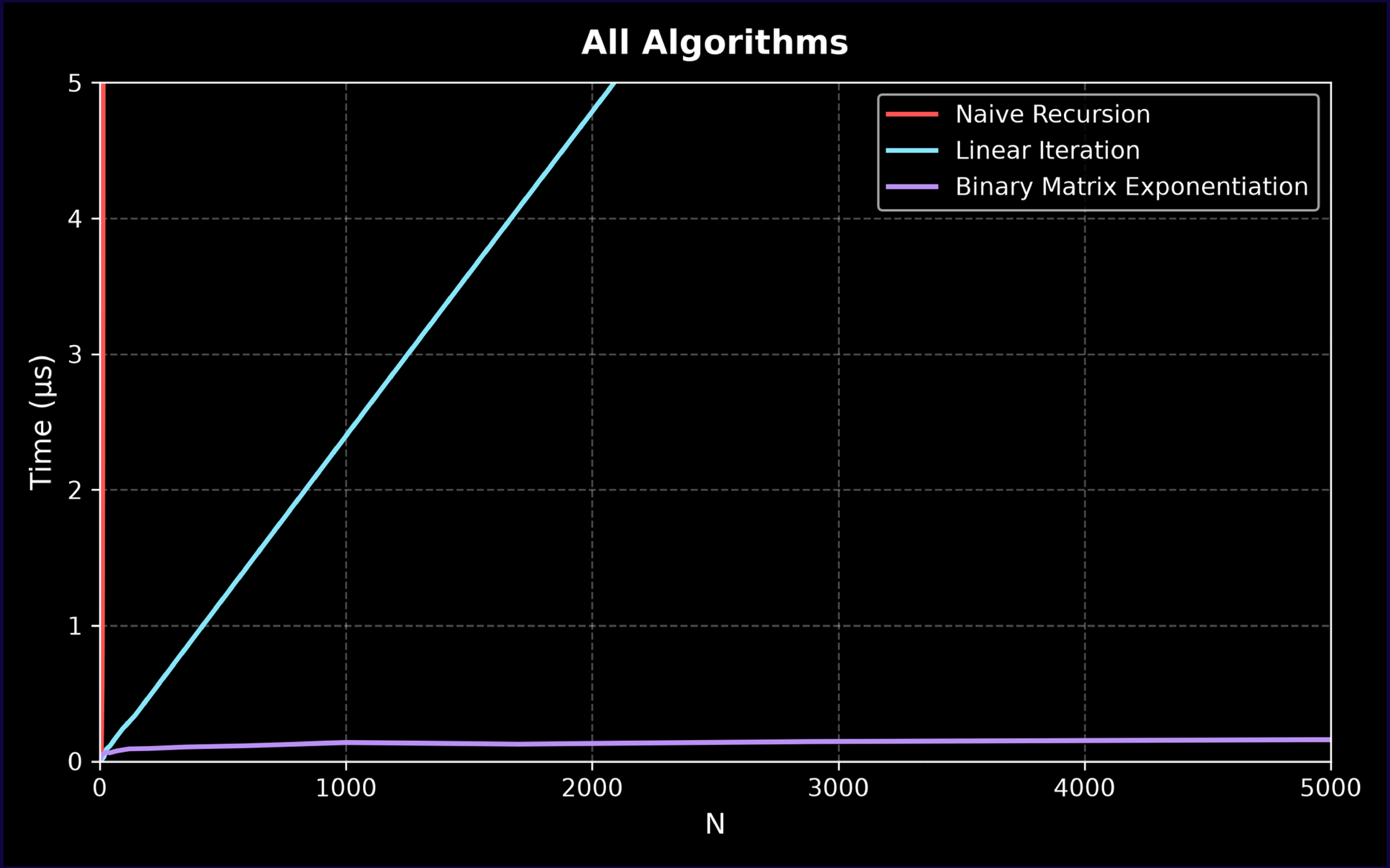
what we have achieved



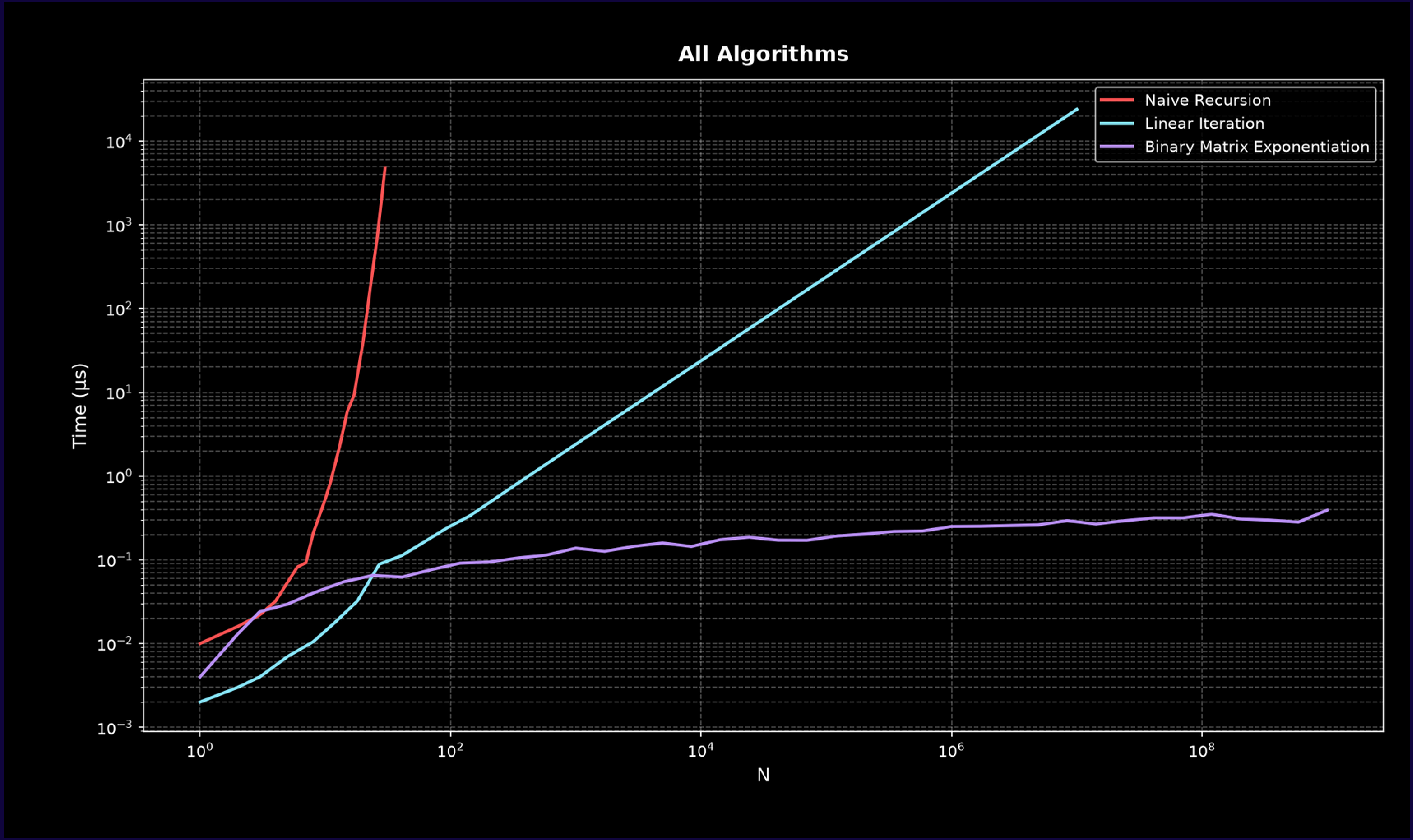
WAIT! WHY IS LINEAR BETTER THAN BINEXP?



phew, that's better
but this looks kinda boring?



displaying it better - say hello to the log scales



THANKS!

ANY QUESTIONS?

join and follow!

(if you haven’t already)

PROGRAMMINGCLUB_IITM